

|                                                                                                                       |                                                                                                                                   |
|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
|  <p><b>Proyecto - Aligest U2</b></p> | Departamento: Ciencias de la Computación                                                                                          |
|                                                                                                                       | Carrera: Ingeniería en Software<br>Asignatura: Análisis y Diseño de Software<br>NRC: 30723                                        |
|                                                                                                                       | Apellidos y nombres del estudiante:<br>Cedeño Cuenca Andrés Isaías<br>Santi Cruz Jeancarlo Javier<br>Tufiño Ortiz Erick Alejandro |

## Índice de Contenidos

|                                                                                            |           |
|--------------------------------------------------------------------------------------------|-----------|
| <b>1.- Diagrama de clases de Análisis.....</b>                                             | <b>2</b>  |
| 1. Código en PlantUML.....                                                                 | 2         |
| 2. Captura del Diagrama generado.....                                                      | 6         |
| <b>2.- Diagrama del Puente para el Módulo “Administrar Sistema” (REQ001 - REQ003).....</b> | <b>7</b>  |
| 1. Código en PlantUML.....                                                                 | 7         |
| 2. Captura del Diagrama Generado.....                                                      | 8         |
| <b>3.- Diagrama del Puente Módulo "Gestionar Copropietarios" (REQ004-REQ008).....</b>      | <b>9</b>  |
| 1. Código en PlantUML.....                                                                 | 9         |
| 2. Captura del Diagrama generado.....                                                      | 10        |
| <b>4.- Diagrama de Clases de Diseño con Arquitectura de Tres Capas.....</b>                | <b>11</b> |
| 1. Código en PlantUML.....                                                                 | 11        |
| 2. Captura del Diagrama Generado.....                                                      | 13        |
| 3. Explicación.....                                                                        | 13        |
| <b>5.- Diagrama de implementación del patrón Adapter.....</b>                              | <b>13</b> |
| 1. Código en PlantUML.....                                                                 | 13        |
| 2. Captura del Diagrama Generado.....                                                      | 16        |
| 3. Justificación.....                                                                      | 16        |
| <b>6.- Diagrama de implementación del patrón Strategy.....</b>                             | <b>17</b> |
| 1. Código en PlantUML.....                                                                 | 17        |
| 2. Captura del Diagrama Generado.....                                                      | 18        |
| 3. Justificación.....                                                                      | 19        |
| <b>7.- Diagrama de implementación del patrón Decorator.....</b>                            | <b>19</b> |
| 1. Código en PlantUML.....                                                                 | 19        |
| 2. Captura del Diagrama Generado.....                                                      | 21        |
| 3. Justificación.....                                                                      | 21        |
| <b>8.- Diagrama de Arquitectura de 3 Capas Integrada + los patrones de diseño.....</b>     | <b>21</b> |
| 1. Código en PlantUML.....                                                                 | 21        |
| 2. Captura del Diagrama Generado.....                                                      | 24        |

# 1.- Diagrama de clases de Análisis

## 1. Código en PlantUML

```
@startuml ClasesAnalisis_SAC
title SAC - Diagrama de Clases de Análisis\nSprint 0, 1 y 2 (REQ001-REQ008)
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam defaultTextAlignment center
skinparam class {
    BackgroundColor #FFFFFF
    BorderColor #444444
    FontColor #000000
}

package "Módulo Administrar Sistema" as ModAdmin #F4F8FB {

    package "Boundary - Capa de Presentación" as BoundaryAdmin #D9EAF7 {
        class VistaLogin <<boundary>> #EAF4FB {
            +mostrarFormularioLogin()
            +capturarCredenciales()
            +mostrarMensajeError(mensaje)
            +redirigirSegunPerfil(perfil)
        }
        class VistaRecuperacionContrasena <<boundary>> #EAF4FB {
            +mostrarFormularioRecuperacion()
            +capturarCorreo()
            +capturarCodigoVerificacion()
            +capturarNuevaContrasena()
            +mostrarMensajeEstado(mensaje)
        }
        class VistaGestionPerfiles <<boundary>> #EAF4FB {
            +mostrarListaUsuarios(usuarios)
            +mostrarFormularioEdicionPerfil(usuario)
            +capturarNuevoPerfil()
            +mostrarMensajeValidacion(mensaje)
        }
    }

    package "Control - Capa de Lógica de Negocio" as ControlAdmin #DFF2E1 {
        class AccesoController <<control>> #F3FBF4 {
            +iniciarSesion(usuario, contrasena)
            +solicitarRecuperacion(correo)
            +validarCodigoYActualizarContrasena(codigo, nuevaContrasena)
        }
        class PerfilController <<control>> #F3FBF4 {
            +listarUsuarios()
            +cambiarPerfil(usuarioId, nuevoPerfil)
        }
        class ServicioAutenticacion <<control>> #F3FBF4 {
            -maxIntentosFallidos: int
            +validarCredenciales(usuario, contrasena): boolean
            +registrarIntentoFallido(usuario)
            +bloquearUsuario(usuario)
            +crearSesion(usuario): SesionActiva
            +determinarRedireccion(perfil): String
        }
        class ServicioRecuperacionContrasena <<control>> #F3FBF4 {
            -tiempoExpiracionMinutos: int
            +validarCorreoExistente(correo): boolean
            +generarCodigoVerificacion(usuario): CodigoVerificacion
            +enviarCodigoPorCorreo(usuario, codigo)
            +validarCodigo(codigo, usuario): boolean
            +actualizarContrasena(usuario, nuevaContrasena)
        }
        class ServicioPerfil <<control>> #F3FBF4 {
            +obtenerUsuarios(): List<Usuario>
            +actualizarPerfil(usuarioId, nuevoPerfil)
            +validarAdministradorActivo(): boolean
        }
    }
}
```

```

package "Entity y Datos" as EntityAdmin #F3E0F7 {
class Usuario <<entity>> #FCF7FD {
    -id: int
    -nombre: String
    -correo: String
    -contrasenaHash: String
    -rol: String
    -intentosFallidos: int
    -estado: String
    +validarPassword(passwordIngresada): boolean
    +incrementarIntentoFallido()
    +resetearIntentos()
    +actualizarContrasena(nuevaHash)
    +cambiarRol(nuevoRol)
}
class CodigoVerificacion <<entity>> #FCF7FD {
    -codigo: String
    -usuarioId: int
    -fechaGeneracion: Date
    -fechaExpiracion: Date
    -usado: boolean
    +estaExpirado(): boolean
    +marcarComoUsado()
}
class RepositorioUsuario <<repository>> #FAF1FC {
    +buscarPorCorreo(correo): Usuario
    +buscarPorId(id): Usuario
    +guardar(usuario)
    +actualizar(usuario)
    +listarTodos(): List<Usuario>
    +contarAdministradoresActivos(): int
}
class RepositorioCodigoVerificacion <<repository>> #FAF1FC {
    +guardar(codigo)
    +buscarPorUsuario(usuarioId): CodigoVerificacion
    +invalidar(codigo)
}
}
}

package "Módulo Gestionar Copropietarios" as ModCopro #F4F8FB {

package "Boundary - Capa de Presentación" as BoundaryCopro #D9EAF7 {
class VistaGestionCopropietarios <<boundary>> #EAF4FB {
    +mostrarFormularioCargaExcel()
    +mostrarTablaCopropietarios(lista)
    +capturarFiltrosBusqueda()
    +mostrarFormularioEdicion(copropietario)
    +mostrarConfirmacionEliminacion(copropietario)
    +mostrarResumenImportacion(resumen)
}
class VistaReportes <<boundary>> #EAF4FB {
    +mostrarFormularioFiltrosReporte()
    +mostrarTablaReporte(datos)
    +exportarResultados(formato)
}
}

package "Control - Capa de Lógica de Negocio" as ControlCopro #DFF2E1 {
class CopropietarioController <<control>> #F3FBF4 {
    +importarDesdeExcel(archivo)
    +consultarCopropietarios(criterio)
    +modificarCopropietario(id, datos)
    +eliminarCopropietario(id)
}
class ReporteController <<control>> #F3FBF4 {
    +generarReporte(filtros, usuarioActual)
    +exportarReportePDF(reporte)
}
class ServicioCopropietario <<control>> #F3FBF4 {
    +importar(archivo): ResumenImportacion
}
}

```

```

+validarFilaExcel(fila): boolean
+detectarDuplicado(cedula): boolean
+generarCredencialesTemporales(copropietario)
+buscar(criterio): List<Copropietario>
+actualizar(copropietario): Copropietario
+validarDatos(copropietario): boolean
+eliminar(id)
+tieneHistorialFinanciero(id): boolean
+invalidarCredenciales(id)
}
class ServicioReportes <<control>> #F3FBF4 {
+aplicarRestriccionPorRol(filtros, usuarioActual): Filtros
+consultarPagosYEstadoCuenta(filtros): List<EstadoCuenta>
+generarPDF(datosReporte): Documento
}
}

package "Entity y Datos" as EntityCopro #F3E0F7 {
class Copropietario <<entity>> #FCF7FD {
-id: int
-cedula: String
-nombre: String
-correo: String
-telefono: String
-numeroCasa: String
-usuarioid: int
-fechaRegistro: Date
+actualizarDatos(datos)
+esDuplicado(otraCedula): boolean
}
class EstadoCuenta <<entity>> #FCF7FD {
-copropietarioid: int
-periodo: String
-montoBase: double
-recargoMora: double
-saldoPendiente: double
-estado: String
+calcularTotal(): double
+estaVencido(): boolean
}
class RepositorioCopropietario <<repository>> #FAF1FC {
+buscarPorCriterio(criterio): List<Copropietario>
+buscarPorCedula(cedula): Copropietario
+guardar(copropietario)
+actualizar(copropietario)
+eliminar(id)
+existeCedula(cedula): boolean
}
class RepositorioEstadoCuenta <<repository>> #FAF1FC {
+buscarPorCopropietario(copropietarioid): List<EstadoCuenta>
+listarPorFiltros(filtros): List<EstadoCuenta>
}
}
}

' ===== Relaciones Módulo Administrar Sistema =====
VistaLogin --> AccesoController : envía credenciales
VistaRecuperacionContrasena --> AccesoController : envía solicitud
VistaGestionPerfiles --> PerfilController : envía cambios

AccesoController --> ServicioAutenticacion : coordina
AccesoController --> ServicioRecuperacionContrasena : coordina
PerfilController --> ServicioPerfil : coordina

ServicioAutenticacion --> Usuario : valida / actualiza intentos
ServicioAutenticacion --> RepositorioUsuario : consulta credenciales
ServicioRecuperacionContrasena --> Usuario : actualiza contraseña
ServicioRecuperacionContrasena --> CodigoVerificacion : genera / valida
ServicioRecuperacionContrasena --> RepositorioUsuario : consulta / actualiza
ServicioRecuperacionContrasena --> RepositorioCodigoVerificacion : guarda / valida
ServicioPerfil --> Usuario : modifica rol
ServicioPerfil --> RepositorioUsuario : consulta / actualiza

```

```

RepositorioUsuario "1" --> "0..*" Usuario : almacena / recupera
RepositorioCodigoVerificacion "1" --> "0..*" CodigoVerificacion : almacena / recupera
CodigoVerificacion "1" --> "1" Usuario : pertenece a

' ===== Relaciones Módulo Gestionar Copropietarios =====
VistaGestionCopropietarios --> CopropietarioController : envía archivo,\nriterios o formularios
VistaReportes --> ReporteController : envía filtros

CopropietarioController --> ServicioCopropietario : coordina operación
ReporteController --> ServicioReportes : coordina generación

ServicioCopropietario --> Copropietario : crea / valida /nactualiza / elimina
ServicioCopropietario --> RepositorioCopropietario : consulta / persiste
ServicioReportes --> Copropietario : consulta datos
ServicioReportes --> EstadoCuenta : consulta saldos y deudas
ServicioReportes --> RepositorioCopropietario : consulta
ServicioReportes --> RepositorioEstadoCuenta : consulta

RepositorioCopropietario "1" --> "0..*" Copropietario : almacena / recupera
RepositorioEstadoCuenta "1" --> "0..*" EstadoCuenta : almacena / recupera
EstadoCuenta "0..*" --> "1" Copropietario : pertenece a

' ===== Relación entre módulos =====
Copropietario "1" --> "1" Usuario : tiene credenciales de acceso
ServicioCopropietario ..> Usuario : crea / invalida credenciales\n(REQ004, REQ008)
ServicioCopropietario ..> RepositorioUsuario : usa

' ===== Notas de trazabilidad hacia el diseño =====
note left of ServicioCopropietario #FFF2CC
Strategy (diseño):
buscar(criterio) seleccionará en
tiempo de ejecución la estrategia
de búsqueda (por casa, nombre,
perfil o estado de cuenta) - REQ005.
end note

note right of ServicioCopropietario #FFF2CC
Decorator (diseño):
importar(), actualizar() y eliminar()
serán envueltos con validación de
datos/historial y auditoría, sin
modificar esta clase base
- REQ004, REQ007, REQ008.
end note

note bottom of ServicioCopropietario #FFF2CC
Adapter (diseño):
importar(archivo) recibirá los datos
ya convertidos desde el formato
Excel externo a Copropietario - REQ004.
end note

note right of ServicioReportes #FFF2CC
Strategy (diseño):
generarReporte() seleccionará el
algoritmo según el tipo de reporte
y el rol del usuario - REQ006.
end note

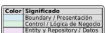
note right of ServicioAutenticacion #FFF2CC
Decorator (diseño - opcional):
Puede decorarse con control de
intentos fallidos y auditoría de
accesos sin alterar la validación
base - REQ001.
end note

note right of Copropietario #FFF2CC
Cada Copropietario está vinculado
a un Usuario que contiene sus
credenciales de acceso

```

@endumll

SAC - Diagrama de Clases de Análisis  
Sprint 0, 1 y 2 (REQ001-REQ008)



## 2.- Diagrama del Puente para el Módulo “Administrar Sistema” (REQ001 - REQ003)

### 1. Código en PlantUML

```
@startuml Puente_CasosUso_Clases_AdministrarSistema
title SAC - Puente de Casos de Uso hacia Clases de Análisis\nMódulo: Administrar Sistema (Sprint 0)\nREQ001, REQ002, REQ003

left to right direction
skinparam shadowing false
skinparam packageStyle rectangle
skinparam rectangleRoundCorner 12
skinparam defaultTextAlignment center
skinparam actor {
    BackgroundColor #D9EAF7
    BorderColor #2F5597
}
skinparam usecase {
    BackgroundColor #EAF4FB
    BorderColor #2F5597
    FontColor #000000
}
skinparam class {
    BackgroundColor #FFFFFF
    BorderColor #444444
    FontColor #000000
}

actor "Administrador" as Admin <<Actor>>
actor "Copropietario" as Copro <<Actor>>

rectangle "SAC - Sprint 0\nMódulo de Administrar Sistema" #F4F8FB {

    usecase "CU-01\nIniciar Sesión\nREQ001" as CU01
    usecase "CU-02\nRecuperar Contraseña\nREQ002" as CU02
    usecase "CU-03\nConfigurar Perfiles\nREQ003" as CU03

    boundary "VistaLogin\n<<Boundary>>\n\n- Captura usuario y contraseña\n- Muestra mensajes de error\n- Redirige según perfil" as VLogin #D9EAF7
    boundary "VistaRecuperacionContrasena\n<<Boundary>>\n\n- Captura correo\n- Captura código de verificación\n- Captura nueva contraseña" as VRecuperacion #D9EAF7
    boundary "VistaGestionPerfiles\n<<Boundary>>\n\n- Lista usuarios y perfiles\n- Formulario de cambio de rol" as VPerfiles #D9EAF7

    control "AccesoController\n<<Control>>\n\n- Recibe credenciales\n- Recibe solicitudes de recuperación\n- Coordina autenticación y recuperación" as AccesoController #DFF2E1
    control "PerfilController\n<<Control>>\n\n- Recibe solicitudes de cambio de perfil\n- Coordina validación de roles" as PerfilController #DFF2E1

    control "ServicioAutenticacion\n<<Control>>\n\n- Valida credenciales\n- Controla intentos fallidos\n- Crea sesión activa" as ServicioAuth #DFF2E1
    control "ServicioRecuperacionContrasena\n<<Control>>\n\n- Genera código de 6 dígitos\n- Valida expiración\n- Actualiza contraseña cifrada" as ServicioRecuperacion #DFF2E1
    control "ServicioPerfil\n<<Control>>\n\n- Cambia rol de usuario\n- Valida administrador activo" as ServicioPerfil #DFF2E1

    entity "Usuario\n<<Entity>>\n\n- id\n- nombre\n- correo\n- contraseaHash\n- rol\n- intentosFallidos\n- estado" as Usuario #F3E0F7
    entity "CodigoVerificacion\n<<Entity>>\n\n- codigo\n- usuarioId\n- fechaExpiracion\n- usado" as CodigoVerif #F3E0F7

    database "RepositorioUsuario\n<<Repository>>\n\n- buscarPorCorreo\n- guardar\n- actualizar" as RepoUsuario #FCE4D6
    database "RepositorioCodigoVerificacion\n<<Repository>>\n\n- guardar\n- buscarPorUsuario\n- invalidar" as RepoCodigo #FCE4D6

    Admin --> CU01
    Admin --> CU02
    Admin --> CU03
    Copro --> CU01
}
```

Copro --> CU02

CU01 ..> VLogin : <<atendido por>>

CU02 ..> VRecuperacion : <<atendido por>>

CU03 ..> VPerfiles : <<atendido por>>

VLLogin --> AccesoController : envía credenciales

VRecuperacion --> AccesoController : envía solicitud de recuperación

VPerfiles --> PerfilController : envía cambios de perfil

AccesoController --> ServicioAuth : coordina inicio de sesión

AccesoController --> ServicioRecuperacion : coordina recuperación

PerfilController --> ServicioPerfil : coordina cambio de perfil

ServicioAuth --> Usuario : valida / actualiza intentos

ServicioAuth --> RepoUsuario : consulta credenciales

ServicioRecuperacion --> Usuario : actualiza contraseña

ServicioRecuperacion -->CodigoVerif : genera / valida

ServicioRecuperacion --> RepoUsuario : consulta / actualiza

ServicioRecuperacion --> RepoCodigo : guarda / valida

ServicioPerfil --> Usuario : modifica rol

ServicioPerfil --> RepoUsuario : consulta / actualiza

RepoUsuario --> Usuario : almacena / recupera

RepoCodigo --> CodigoVerif : almacena / recupera

}

legend right

|= Color |= Elemento |

<#EAF4FB>| Casos de Uso |

<#D9EAF7>| Boundary / Presentación |

<#DFF2E1>| Control / Lógica de negocio |

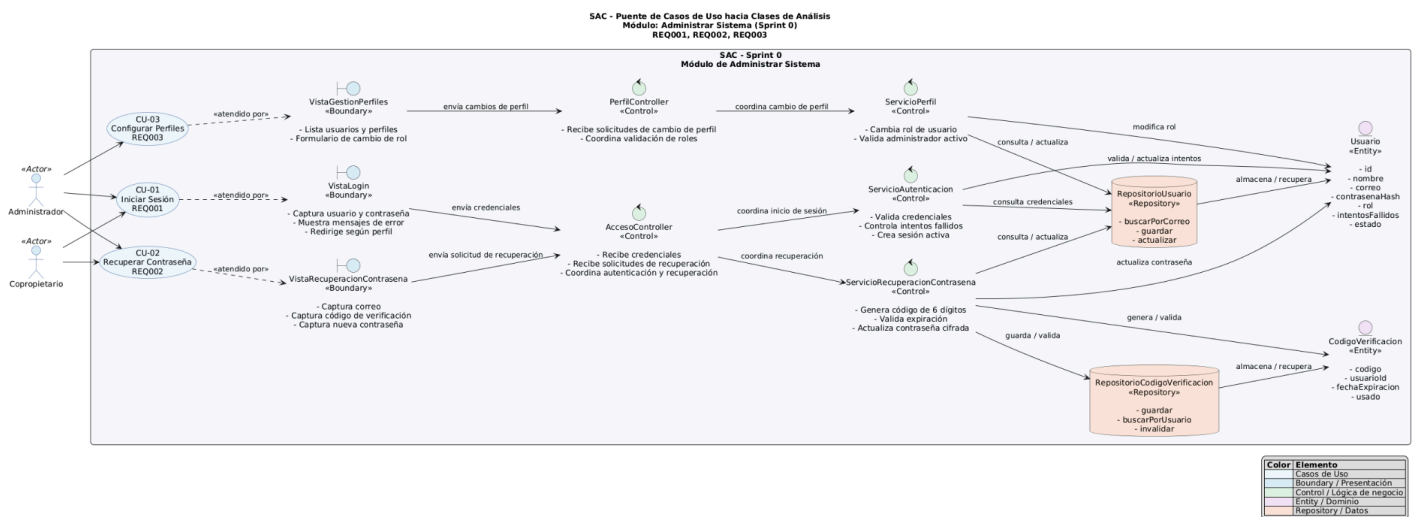
<#F3E0F7>| Entity / Dominio |

<#FCE4D6>| Repository / Datos |

endlegend

@enduml

## 2. Captura del Diagrama Generado



### 3.- Diagrama del Puente Módulo "Gestionar Copropietarios" (REQ004-REQ008)

#### 1. Código en PlantUML

```
@startuml
    Puente_CasosUso_Clases_GestionarCopropietarios
    title SAC - Puente de Casos de Uso hacia Clases de Análisis
    Módulo: Gestionar Copropietarios (Sprint 1 y 2)
    REQ004, REQ005, REQ006, REQ007, REQ008

    left to right direction
    skinparam shadowing false
    skinparam packageStyle rectangle
    skinparam rectangleRoundCorner 12
    skinparam defaultTextAlignment center
    skinparam actor {
        BackgroundColor #D9EAF7
        BorderColor #2F5597
    }
    skinparam usecase {
        BackgroundColor #EAF4FB
        BorderColor #2F5597
        FontColor #000000
    }
    skinparam class {
        BackgroundColor #FFFFFF
        BorderColor #444444
        FontColor #000000
    }

    actor "Administrador" as Admin <<Actor>>
    actor "Copropietario" as Copro <<Actor>>

    rectangle "SAC - Sprints 1 y 2"
        Módulo de Gestionar Copropietarios
        usecase "CU-04\nImportar Copropietarios\nREQ004" as CU04
        usecase "CU-05\nConsultar Copropietarios\nREQ005" as CU05
        usecase "CU-06\nGenerar Reportes\nREQ006" as CU06
        usecase "CU-07\nModificar Copropietarios\nREQ007" as CU07
        usecase "CU-08\nEliminar Copropietarios\nREQ008" as CU08

        boundary "VistaGestionCopropietarios\n<<Boundary>>\n- Carga archivo Excel\n- Muestra tabla con filtros\n- Formularios de edición\n- Confirmación de eliminación" as VGestion
        boundary "VistaReportes\n<<Boundary>>\n- Filtros de fecha/período/estado\n- Muestra tabla de resultados\n- Botón de exportar PDF" as VReportes

        control "CopropietarioController\n<<Control>>\n- Recibe archivo, criterios,\n- formularios y confirmaciones\n- Coordina importación, consulta,\n- modificación y eliminación" as CoproController
        control "ReporteController\n<<Control>>\n- Recibe filtros de reporte\n- Coordina generación y exportación" as ReporteController

        control "ServicioCopropietario\n<<Control>>\n- importar(archivo)\n- buscar(criterio)\n- actualizar(copropietario)\n- eliminar(id)" as ServicioCopro
        control "ServicioReportes\n<<Control>>\n- generarReporte(filtros, rolUsuario)\n- exportarPDF(reporte)" as ServicioReportes

        entity "Copropietario\n<<Entity>>\n- id\n- cedula\n- nombre\n- correo\n- telefono\n- numeroCasa\n- perfil\n- estadoCuenta" as Copropietario
        entity "EstadoCuenta\n<<Entity>>\n- copropietarioId\n- periodos\n- saldoPendiente\n- estado" as EstadoCuenta

        database "RepositorioCopropietario\n<<Repository>>\n- buscarPorCriterio\n- guardar\n- actualizar\n- eliminar\n- existeCedula" as RepoCopro
        database "RepositorioEstadoCuenta\n<<Repository>>\n- buscarPorCopropietario\n- listarPorFiltros" as RepoEstado

        Admin --> CU04
        Admin --> CU05
        Admin --> CU06
        Copro --> CU06
        Admin --> CU07
        Admin --> CU08

        CU04 ..> VGestion : <<atendido por>>
    end
end
```

CU05 ..> V Gestion : <<atendido por>>  
 CU07 ..> V Gestion : <<atendido por>>  
 CU08 ..> V Gestion : <<atendido por>>  
 CU06 ..> V Reportes : <<atendido por>>

V Gestion --> CoproController : envía archivo, \ncriterios o formularios  
 CoproController --> ServicioCopro : coordina operación

V Reportes --> ReporteController : envía filtros  
 ReporteController --> ServicioReportes : coordina generación

ServicioCopro --> Copropietario : crea / valida / \nactualiza / elimina  
 ServicioCopro --> RepoCopro : consulta o persiste

ServicioReportes --> Copropietario : consulta datos  
 ServicioReportes --> EstadoCuenta : consulta saldos y deudas  
 ServicioReportes --> RepoCopro : consulta  
 ServicioReportes --> RepoEstado : consulta

RepoCopro --> Copropietario : almacena / recupera  
 RepoEstado --> EstadoCuenta : almacena / recupera  
 }

legend right

|= Color |= Elemento |

<#EAF4FB>| Casos de Uso |

<#D9EAF7>| Boundary / Presentación |

<#DFF2E1>| Control / Lógica de negocio |

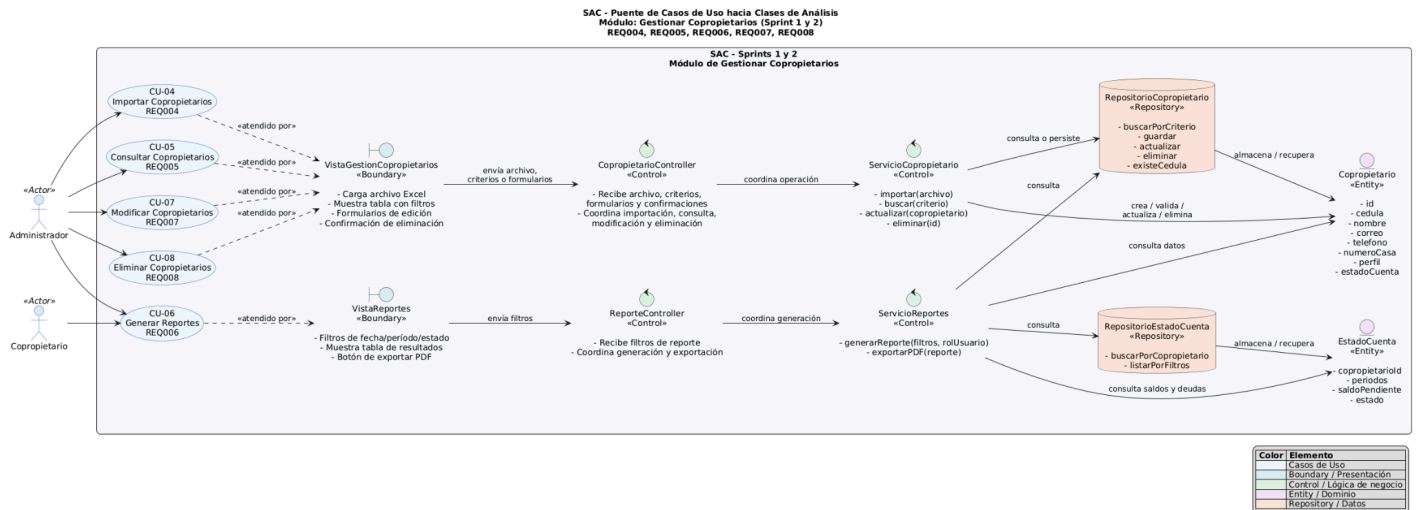
<#F3E0F7>| Entity / Dominio |

<#FCE4D6>| Repository / Datos |

endlegend

@enduml

## 2. Captura del Diagrama generado



## 4.- Diagrama de Clases de Diseño con Arquitectura de Tres Capas

### 1. Código en PlantUML

```
@startuml
    DisenoClases_ArquitecturaTresCapas_SAC
    title SAC - Diagrama de Clases de Diseño\nArquitectura de Tres Capas (REQ001-REQ008)
    top to bottom direction
    skinparam shadowing false
    skinparam packageStyle rectangle
    skinparam rectangleRoundCorner 12
    skinparam defaultTextAlignment center
    skinparam class {
        BackgroundColor #FFFFFF
        BorderColor #444444
        FontColor #000000
    }

    actor "Administrador \nCopropietario" as ActorUsuario

    rectangle "CAPA DE PRESENTACIÓN" as CAPA_PRESENTACION #D9EAF7 {
        rectangle "Administrar Sistema" as PresAdmin #EAF4FB {
            class VistaLogin <<View>> {}
            class VistaRecuperacionContrasena <<View>> {}
            class VistaGestionPerfiles <<View>> {}
            class AccesoController <<Controller>> {}
            class PerfilController <<Controller>> {}
        }
        rectangle "Gestionar Copropietarios" as PresCopro #EAF4FB {
            class VistaGestionCopropietarios <<View>> {}
            class VistaReportes <<View>> {}
            class CopropietarioController <<Controller>> {}
            class ReporteController <<Controller>> {}
        }
    }

    rectangle "CAPA DE LÓGICA DE NEGOCIO" as CAPA_NEGOCIO #DFF2E1 {
        rectangle "Servicios - Administrar Sistema" as NegAdmin #F3FBF4 {
            class ServicioAutenticacion <<Service>> {}
            class ServicioRecuperacionContrasena <<Service>> {}
            class ServicioPerfil <<Service>> {}
        }
        rectangle "Servicios - Gestionar Copropietarios" as NegCopro #F3FBF4 {
            class ServicioCopropietario <<Service>> {}
            class ServicioReportes <<Service>> {}
        }
        rectangle "Entidades de Dominio" as Dominio #FCF7FD {
            class Usuario <<Entity>> {}
            classCodigoVerificacion <<Entity>> {}
            class Copropietario <<Entity>> {}
            class EstadoCuenta <<Entity>> {}
        }
    }

    rectangle "CAPA DE DATOS" as CAPA_DATOS #F3E0F7 {
        rectangle "Repositorios" as Repos #FAF1FC {
            class RepositorioUsuario <<Repository>> {}
            class RepositorioCodigoVerificacion <<Repository>> {}
            class RepositorioCopropietario <<Repository>> {}
            class RepositorioEstadoCuenta <<Repository>> {}
        }
        database "Base de Datos\nSAC" as BD
    }

    ActorUsuario --> PresAdmin : interactúa
    ActorUsuario --> PresCopro : interactúa

    PresAdmin --> NegAdmin : invoca servicios
    PresCopro --> NegCopro : invoca servicios

    NegAdmin --> Dominio : usa / manipula
    NegCopro --> Dominio : usa / manipula
```

NegAdmin --> Repos : consulta / persiste  
NegCopro --> Repos : consulta / persiste

' Dependencia cruzada relevante de la arquitectura  
ServicioCopropietario --> Usuario : crea / invalida\ncredenciales  
ServicioCopropietario --> RepositorioUsuario : usa (REQ004, REQ008)

Repos --> BD : ejecuta consultas

note right of CAPA\_PRESENTACION #FFF2CC

Capa de Presentación:

- Recibe la interacción del usuario.
- Solo conoce a la Capa de Lógica de Negocio (Servicios).
- No accede directamente a Repositorios ni a la Base de Datos.

end note

note right of CAPA\_NEGOCIO #FFF2CC

Capa de Lógica de Negocio:

- Contiene Servicios (reglas de negocio, validaciones, cálculos) y Entidades de Dominio.
- Aquí se ubicarán los patrones Adapter, Strategy y Decorator (ver diagrama de arquitectura integrada).

end note

note right of CAPA\_DATOS #FFF2CC

Capa de Datos:

- Los Repositorios encapsulan el acceso a la Base de Datos.
- No conoce a las capas superiores (dependencia en un solo sentido).

end note

legend right

|= Color |= Capa |

|<#D9EAF7>| Presentación (Vistas + Controllers) |

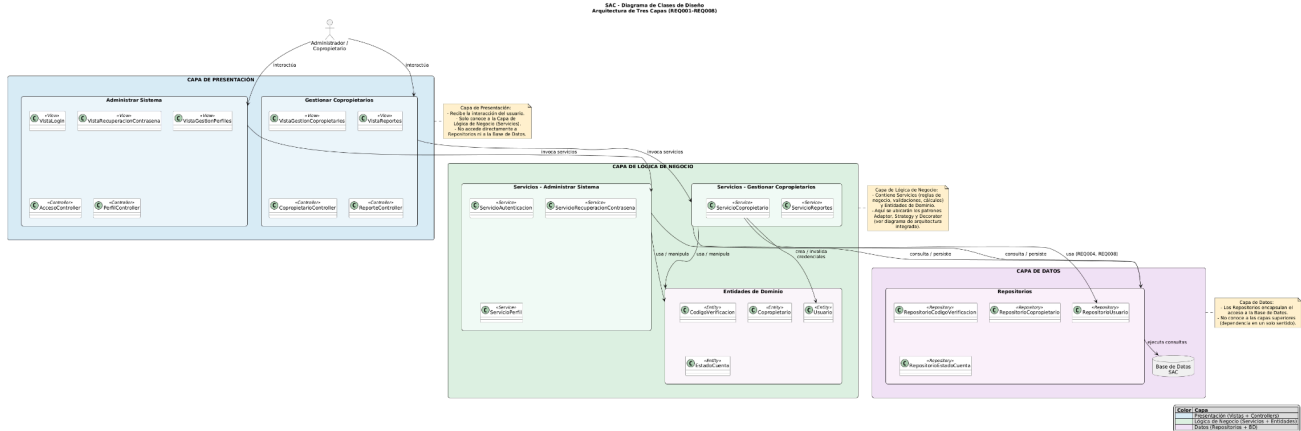
|<#DFF2E1>| Lógica de Negocio (Servicios + Entidades) |

|<#F3E0F7>| Datos (Repositorios + BD) |

endlegend

@enduml

## 2. Captura del Diagrama Generado



## 3. Explicación

Consideramos que la arquitectura de tres capas se adapta adecuadamente al proyecto AliGest ya que nos permite separar las responsabilidades que cambian por diversos motivos. En la capa de presentación se ubican las interfaces del sistema, como el inicio de sesión, recuperación de contraseña, gestión de perfiles, importación, consulta, reportes, modificación y eliminación de copropietarios. Estas pantallas pueden cambiar por criterios visuales o de usabilidad sin afectar la lógica interna del sistema.

En la capa de lógica de negocio se concentran las reglas principales del sistema, como validar credenciales, mantener al menos un Administrador activo, controlar los permisos por rol, validar datos importados, restringir la información visible para cada usuario y controlar la eliminación de copropietarios con historial financiero. Esta capa permite que las reglas del condominio se mantengan centralizadas y no se repitan en cada pantalla.

Finalmente, la capa de datos se encarga de la persistencia mediante repositorios, evitando que las vistas accedan directamente a la base de datos. Esto facilita futuros cambios, como agregar nuevos repositorios para pagos, estados de cuenta, comprobantes o notificaciones en los siguientes sprints. Por estas diversas razones, nosotros consideramos que la arquitectura de tres capas no solo organiza el diseño actual, sino que también deja preparado el sistema para incorporar los requisitos posteriores sin romper la estructura existente.

## 5.- Diagrama de implementación del patrón Adapter

### 1. Código en PlantUML

```
@startuml
    Adapter_ImportacionCopropietarios
    title SAC - Patrón Adapter aplicado a la Importación de Copropietarios (REQ004)
    skinparam shadowing false
    skinparam classAttributeIconSize 0
    skinparam defaultTextAlignment center
    skinparam class {
        BackgroundColor #FFFFFF
        BorderColor #444444
    }

    package "Datos Externos" #FFF2CC {
        class FilaExcelCopropietario <<Adaptee>> #FFE6E6 {
            -cedulaTexto: String
            -nombreCompletoTexto: String
        }
    }
```

```

-correoTexto: String
-telefonoTexto: String
-casaTexto: String
}
}

package "Capa de Lógica de Negocio\nAplicación del patrón Adapter" #DFF2E1 {

interface IFuenteCopropietario <<Target>> #DCEBFA {
+obtenerCedula(): String
+obtenerNombre(): String
+obtenerCorreo(): String
+obtenerTelefono(): String
+obtenerNumeroCasa(): String
}

class AdaptadorExcelCopropietario <<Adapter>> #FFE6CC {
-fila: FilaExcelCopropietario
+obtenerCedula(): String
+obtenerNombre(): String
+obtenerCorreo(): String
+obtenerTelefono(): String
+obtenerNumeroCasa(): String
+convertirACopropietario(): Copropietario
}

class ServicioCopropietario <<Service>> #F3FBF4 {
+importar(filas: List<IFuenteCopropietario>): ResumenImportacion
+validarFilaExcel(fuente: IFuenteCopropietario): boolean
+detectarDuplicado(cedula): boolean
+generarCredencialesTemporales(copropietario)
}
}

package "Entidades de Dominio" #F3E0F7 {
class Copropietario <<Entity>> #FCF7FD {
-cedula: String
-nombre: String
-correo: String
-telefono: String
-numeroCasa: String
}
}

IFuenteCopropietario <|.. AdaptadorExcelCopropietario
AdaptadorExcelCopropietario o--> FilaExcelCopropietario : envuelve\n(fila original)
AdaptadorExcelCopropietario ..> Copropietario : crea / construye
ServicioCopropietario --> IFuenteCopropietario : recibe datos\nya adaptados
ServicioCopropietario --> Copropietario : valida / registra

note right of FilaExcelCopropietario #FFF2CC
Adaptee:
representa una fila cruda
del archivo Excel, con
formato externo (texto plano).
end note

note right of IFuenteCopropietario #FFF2CC
Target:
interfaz que ServicioCopropietario
espera para leer datos de
un copropietario, sin importar
su origen.
end note

note right of AdaptadorExcelCopropietario #FFF2CC
Adapter:
convierte cada FilaExcelCopropietario
en un objeto compatible con
IFuenteCopropietario y permite
construir un Copropietario válido.
end note

```

note bottom of ServicioCopropietario #FFF2CC

Uso (REQ004):

por cada fila del Excel, el

controlador crea un

AdaptadorExcelCopropietario y lo

entrega a importar(), sin que el

servicio conozca el formato Excel.

end note

legend right

|= Color |= Significado |

|<#FFE6E6>| Adaptee (formato externo) |

|<#DCEBFA>| Target (interfaz esperada) |

|<#FFE6CC>| Adapter |

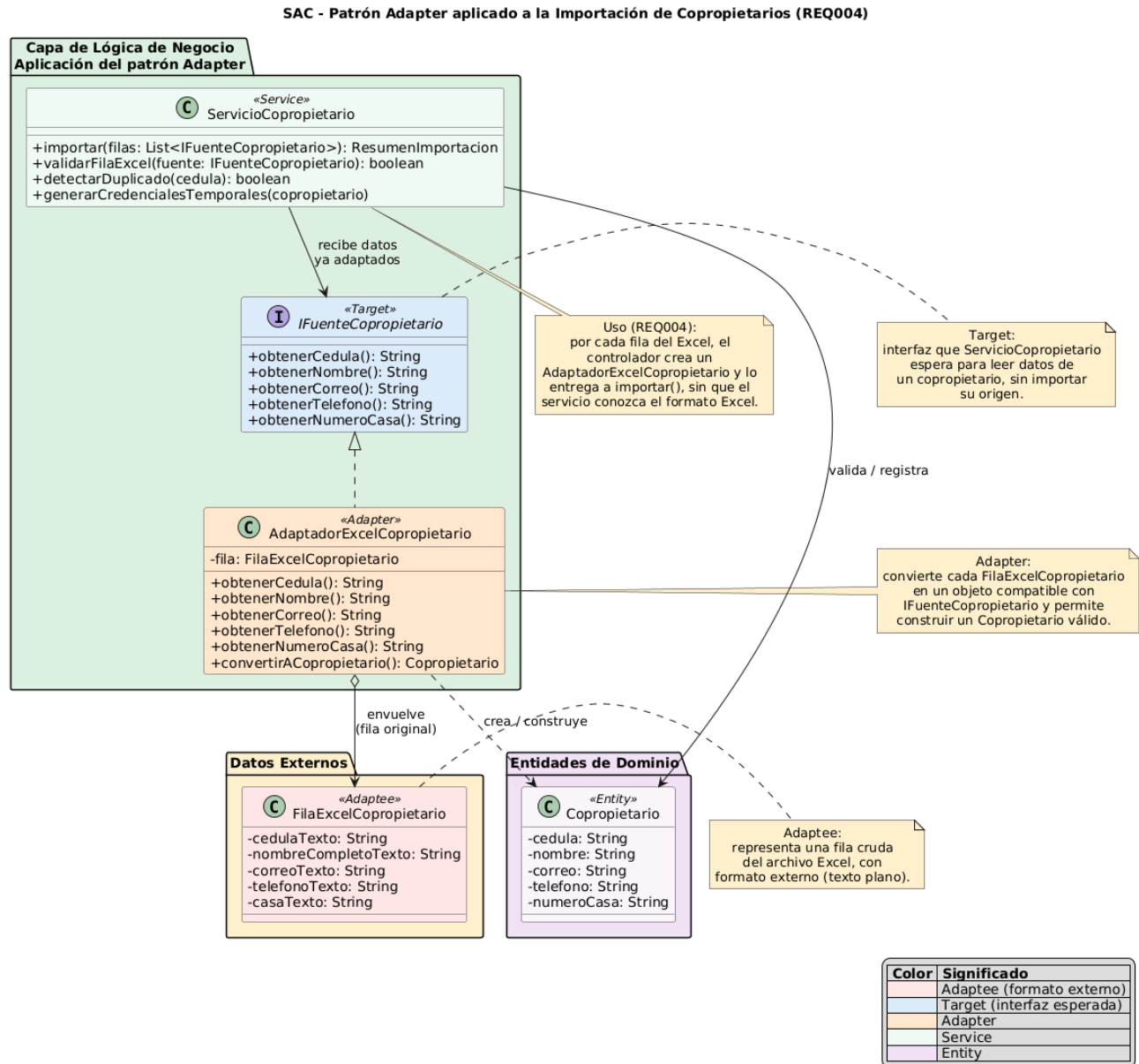
|<#F3FBF4>| Service |

|<#F3E0F7>| Entity |

endlegend

@enduml

## 2. Captura del Diagrama Generado



## 3. Justificación

Decidimos aplicar el patrón Adapter principalmente en la importación de copropietarios desde Excel, correspondiente al REQ004. Este patrón nos permite transformar los datos externos del archivo, como por ejemplo: nombres, cédulas, correos, teléfonos y números de casa, al formato interno que necesita la entidad Copropietario. De esta manera la lógica de negocio no depende directamente del formato del Excel.

Su uso es importante porque nos permite mantener limpio el ServicioCopropietario y separar la conversión de datos externos de las reglas internas del sistema. Además si en el futuro se desea importar desde CSV, una API u otra fuente, solo será necesario crear un nuevo adaptador sin modificar el servicio principal.

## 6.- Diagrama de implementación del patrón Strategy

### 1. Código en PlantUML

```
@startuml
    Strategy_BusquedaYReportesCopropietarios
    title SAC - Patrón Strategy aplicado a Consulta (REQ005) y Reportes (REQ006)
    skinparam shadowing false
    skinparam classAttributeIconSize 0
    skinparam defaultTextAlignment center
    skinparam class {
        BackgroundColor #FFFFFF
        BorderColor #444444
    }

    package "Capa de Lógica de Negocio\nStrategy - Búsqueda (REQ005)" #DFF2E1 {

        interface IEstrategiaBusqueda <<Strategy>> #DCEBFA {
            +buscar(criterio: CriterioBusqueda): List<Copropietario>
        }
        class BusquedaPorCasa <<ConcreteStrategy>> #E8F4FD {
            +buscar(criterio): List<Copropietario>
        }
        class BusquedaPorNombre <<ConcreteStrategy>> #E8F4FD {
            +buscar(criterio): List<Copropietario>
        }
        class BusquedaPorEstadoCuenta <<ConcreteStrategy>> #E8F4FD {
            +buscar(criterio): List<Copropietario>
        }
        class BusquedaPorPerfil <<ConcreteStrategy>> #E8F4FD {
            +buscar(criterio): List<Copropietario>
        }
        class ContextoBusquedaCopropietario <<Context>> #F3FBF4 {
            -estrategiaActual: IEstrategiaBusqueda
            +establecerEstrategia(estrategia: IEstrategiaBusqueda)
            +ejecutarBusqueda(criterio: CriterioBusqueda): List<Copropietario>
        }
    }

    package "Capa de Lógica de Negocio\nStrategy - Reportes (REQ006)" #DFF2E1 {

        interface IEstrategiaReporte <<Strategy>> #DCEBFA {
            +generar(filtros: FiltrosReporte): Reporte
        }
        class ReportePagos <<ConcreteStrategy>> #E8F4FD {
            +generar(filtros): Reporte
        }
        class ReporteMorosidad <<ConcreteStrategy>> #E8F4FD {
            +generar(filtros): Reporte
        }
        class ReporteEstadoCuenta <<ConcreteStrategy>> #E8F4FD {
            +generar(filtros): Reporte
        }
        class ContextoReporte <<Context>> #F3FBF4 {
            -estrategiaActual: IEstrategiaReporte
            +establecerEstrategia(estrategia: IEstrategiaReporte)
            +ejecutarReporte(filtros: FiltrosReporte): Reporte
        }
    }

    class ServicioCopropietario <<Service>> #F3FBF4 {
        +buscar(criterio: CriterioBusqueda): List<Copropietario>
    }
    class ServicioReportes <<Service>> #F3FBF4 {
        +generarReporte(filtros: FiltrosReporte, rolUsuario): Reporte
    }

    IEstrategiaBusqueda <|.. BusquedaPorCasa
    IEstrategiaBusqueda <|.. BusquedaPorNombre
    IEstrategiaBusqueda <|.. BusquedaPorEstadoCuenta
    IEstrategiaBusqueda <|.. BusquedaPorPerfil
    ContextoBusquedaCopropietario o--> IEstrategiaBusqueda : delega búsqueda
```

IEstrategiaReporte <|.. ReportePagos  
 IEstrategiaReporte <|.. ReporteMorosidad  
 IEstrategiaReporte <|.. ReporteEstadoCuenta  
 ContextoReporte o--> IEstrategiaReporte : delega generación

ServicioCopropietario --> ContextoBusquedaCopropietario : usa  
 ServicioReportes --> ContextoReporte : usa

note top of ContextoBusquedaCopropietario #FFF2CC

Strategy (REQ005):

según el filtro elegido en  
 VistaGestionCopropietarios  
 (casa, nombre, perfil o estado  
 de cuenta), el controlador  
 selecciona la estrategia  
 correspondiente en tiempo  
 de ejecución.  
 end note

note top of ContextoReporte #FFF2CC

Strategy (REQ006):

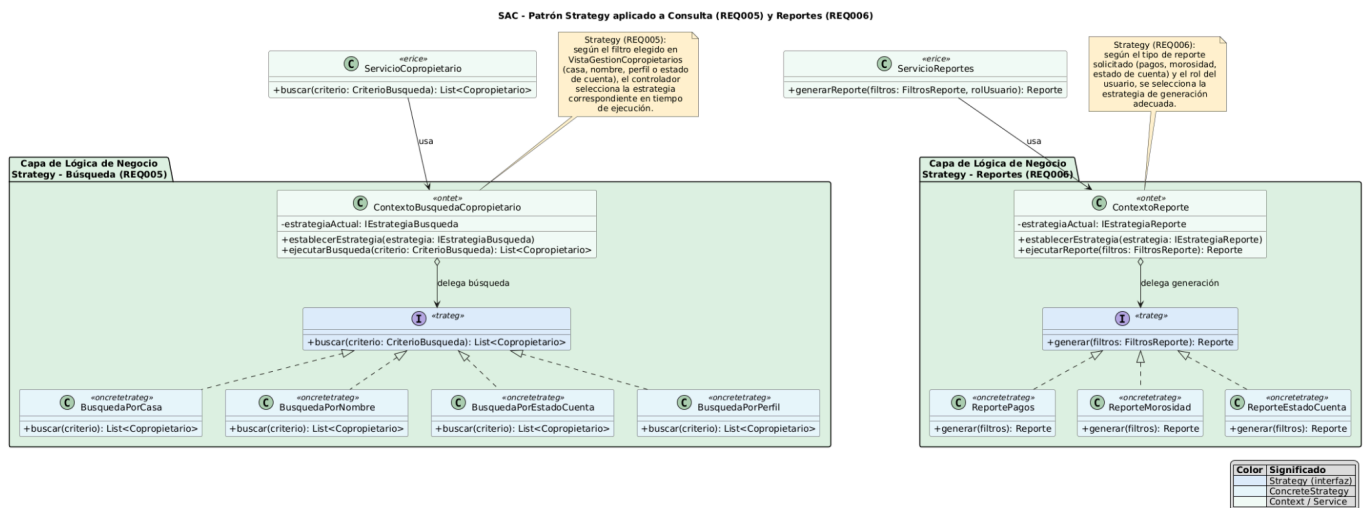
según el tipo de reporte  
 solicitado (pagos, morosidad,  
 estado de cuenta) y el rol del  
 usuario, se selecciona la  
 estrategia de generación  
 adecuada.  
 end note

legend right

|= Color |= Significado |  
 <#DCEBFA> Strategy (interfaz) |  
 <#E8F4FD> ConcreteStrategy |  
 <#F3FBF4> Context / Service |  
 endlegend

@enduml

## 2. Captura del Diagrama Generado



### 3. Justificación

El patrón Strategy lo aplicamos en los requisitos relacionados con consulta y reportes, principalmente en los REQ005 y REQ006. En estos casos, el sistema necesita trabajar con diferentes formas de búsqueda y generación de información, por ejemplo, buscar por número de casa, por nombre, perfil o estado de cuenta, así como generar reportes de pagos, morosidad o estados de cuenta.

Este patrón evita que el sistema dependa de una sola clase con muchas condiciones. En su lugar, cada forma de búsqueda o reporte se implementa como una estrategia independiente. Esto nos permite agregar nuevos filtros o nuevos tipos de reportes en el futuro sin modificar la lógica principal, así mantenemos el sistema más ordenado, extensible y fácil de probar.

## 7.- Diagrama de implementación del patrón Decorator

### 1. Código en PlantUML

```
@startuml Decorator_ServicioCopropietario
title SAC - Patrón Decorator aplicado al Servicio de Copropietarios (REQ004, REQ007, REQ008)
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam defaultTextAlignment center
skinparam class {
    BackgroundColor #FFFFFF
    BorderColor #444444
}

package "Capa de Lógica de Negocio\nAplicación del patrón Decorator" #DFF2E1 {

    interface IServicioCopropietario <<Component>> #DCEBFA {
        +importar(filas: List<IFuenteCopropietario>): ResumenImportacion
        +buscar(criterio: CriterioBusqueda): List<Copropietario>
        +actualizar(copropietario: Copropietario): Copropietario
        +eliminar(id: int)
    }

    class ServicioCopropietarioBase <<ConcreteComponent>> #F3FBF4 {
        +importar(filas): ResumenImportacion
        +buscar(criterio): List<Copropietario>
        +actualizar(copropietario): Copropietario
        +eliminar(id)
    }

    abstract class ServicioCopropietarioDecorator <<Decorator>> #FFE6CC {
        -servicio: IServicioCopropietario
        +importar(filas): ResumenImportacion
        +buscar(criterio): List<Copropietario>
        +actualizar(copropietario): Copropietario
        +eliminar(id)
    }

    class ValidacionCopropietarioDecorator <<ConcreteDecorator>> #FCE5CD {
        +importar(filas): ResumenImportacion
        +actualizar(copropietario): Copropietario
        +eliminar(id)
        -validarDatos(copropietario): boolean
        -validarHistorialFinanciero(id): boolean
    }

    class AuditoriaCopropietarioDecorator <<ConcreteDecorator>> #FCE5CD {
        +importar(filas): ResumenImportacion
        +actualizar(copropietario): Copropietario
        +eliminar(id)
        -registrarAuditoria(accion, detalle)
    }
}

enduml
```

```
IServicioCopropietario <|.. ServicioCopropietarioBase
IServicioCopropietario <|.. ServicioCopropietarioDecorator
ServicioCopropietarioDecorator o--> IServicioCopropietario : envuelve
ServicioCopropietarioDecorator <|-- ValidacionCopropietarioDecorator
ServicioCopropietarioDecorator <|-- AuditoriaCopropietarioDecorator
```

note right of IServicioCopropietario #FFF2CC

Component:

define las operaciones sobre  
copropietarios (REQ004, REQ007,  
REQ008) que usará el controlador.  
end note

note right of ServicioCopropietarioBase #FFF2CC

ConcreteComponent:

contiene la lógica base de  
importar, actualizar y eliminar,  
sin validaciones extra ni auditoría.  
end note

note right of ServicioCopropietarioDecorator #FFF2CC

Decorator:

permite envolver el servicio base  
(o a otro decorador) sin modificar  
su código, agregando capas de  
responsabilidad de forma flexible.  
end note

note right of ValidacionCopropietarioDecorator #FFF2CC

Agrega validación (REQ004, REQ007, REQ008):

- valida cédula, correo, teléfono  
y número de casa antes de  
importar/actualizar.
- antes de eliminar, valida si  
existe historial financiero  
y emite advertencia.

end note

note right of AuditoriaCopropietarioDecorator #FFF2CC

Agrega auditoría:

registra quién y cuándo  
importó, modificó o eliminó  
un copropietario, para  
trazabilidad del módulo.  
end note

note bottom of ServicioCopropietarioDecorator #FFF2CC

Composición típica:

CopropietarioController usará  
new AuditoriaCopropietarioDecorator(  
  new ValidacionCopropietarioDecorator(  
    new ServicioCopropietarioBase()))  
end note

legend right

|= Color |= Significado |

<#DCEBFA>| Component |

<#F3FBF4>| ConcreteComponent |

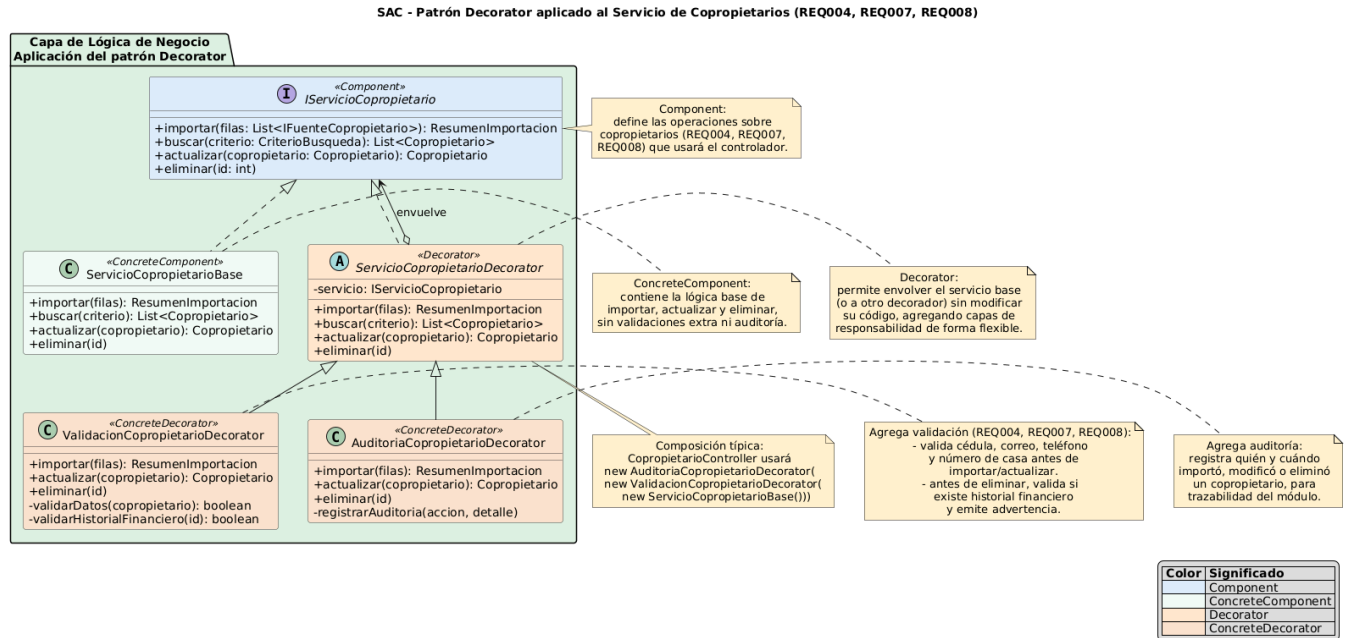
<#FFE6CC>| Decorator |

<#FCE5CD>| ConcreteDecorator |

endlegend

@enduml

## 2. Captura del Diagrama Generado



## 3. Justificación

Este patrón lo aplicamos en las operaciones de gestión de copropietarios, especialmente en REQ004, REQ007 y REQ008. Estas operaciones requieren validaciones, control de historial financiero y auditoría, pero dichas tareas no deberían mezclarse directamente con el CRUD principal.

Con Decorator, el ServicioCopropietarioBase se encarga de las operaciones principales, mientras que clases decoradoras pueden agregar validaciones o auditorías sin modificar la clase base. Por ejemplo, un decorador puede validar datos antes de importar o modificar, y otro puede registrar quién realizó una acción sobre los copropietarios. Esto mejora la separación de responsabilidades y permite extender el sistema en futuros sprints sin reescribir el servicio principal.

## 8.- Diagrama de Arquitectura de 3 Capas Integrada + los patrones de diseño

### 1. Código en PlantUML

```
@startuml SAC_Arquitectura_Adapter_Strategy_Decorator
title Arquitectura integrada de Gestionar Copropietarios - SAC
Tres capas + Adapter + Strategy + Decorator
(REQ004-REQ008)

left to right direction
skinparam shadowing false
skinparam defaultTextAlignment center
skinparam rectangleRoundCorner 12
skinparam componentStyle rectangle
skinparam packageStyle rectangle

actor "Administrador" as Admin
actor "Copropietario" as Copro

cloud "Datos externos\nArchivo Excel (.xlsx)" as Externo #FFF2CC

rectangle "Capa de Presentación" as CAPA_PRESENTACION #D9EAF7 {
    rectangle "VistaGestionCopropietarios\nFormulario / Tabla CRUD" as VistaGestion #EAF4FB
    rectangle "VistaReportes\nFiltros / Tabla de resultados" as VistaReportes #EAF4FB
}
```

```

}

rectangle "Capa de Lógica de Negocio" as CAPA_NEGOCIO #DFF2E1 {
  rectangle "AuditoriaCopropietarioDecorator\n(Decorator)" as Auditoria #FFE6CC
  rectangle "ValidacionCopropietarioDecorator\n(Decorator)" as Validacion #FFE6CC
  rectangle "AdaptadorExcelCopropietario\n(Adapter)" as Adapter #FFE6E6
  rectangle "ServicioCopropietarioBase\n(Servicio base)" as Servicio #F3FBF4
  rectangle "ContextoBusquedaCopropietario\n(Strategy Context)" as CtxBusqueda #DCEBFA
  rectangle "BusquedaPorCasa" as EstrategiaCasa #E8F4FD
  rectangle "BusquedaPorNombre" as EstrategiaNombre #E8F4FD
  rectangle "ServicioReportes\n(Servicio)" as ServicioReportes #F3FBF4
  rectangle "ContextoReporte\n(Strategy Context)" as CtxReporte #DCEBFA
}

rectangle "Capa de Datos" as CAPA_DATOS #F3E0F7 {
  rectangle "RepositorioCopropietario" as RepoCopro #FAF1FC
  rectangle "RepositorioEstadoCuenta" as RepoEstado #FAF1FC
  rectangle "RepositorioUsuario\n(Módulo Administrar Sistema)" as RepoUsuario #FAF1FC
  database "Base de Datos\nSAC" as FuenteDatos #FFFFFF
  rectangle "Copropietario\n(cedula, nombre, correo,\ntelefono, numeroCasa)" as EntidadCopro #FCF7FD
  rectangle "EstadoCuenta\n(periodo, montoBase,\nrecargoMora, saldo)" as EntidadEstado #FCF7FD
}

Admin --> VistaGestion : interacción
Admin --> VistaReportes : interacción
Copro --> VistaReportes : interacción

VistaGestion --> Auditoria : solicitudes CRUD\n(importar, actualizar,\neliminar, buscar)
Auditoria --> Validacion : delega y extiende\n(auditoría)
Validacion --> Servicio : delega y extiende\n(validación)

Adapter --> Servicio : convierte filas Excel\na Copropietario
Externo --> Adapter : cedula, nombre, correo,\ntelefono, numeroCasa

Servicio --> CtxBusqueda : buscar(criterio)
CtxBusqueda --> EstrategiaCasa : estrategia activa
CtxBusqueda --> EstrategiaNombre : estrategia opcional

Servicio --> RepoCopro : guardar / listar /\nactualizar / eliminar
Servicio ..> RepoUsuario : crea / invalida\ncredenciales

VistaReportes --> ServicioReportes : solicita reporte
ServicioReportes --> CtxReporte : generar(filtros, rol)
ServicioReportes --> RepoCopro : consulta
ServicioReportes --> RepoEstado : consulta

RepoCopro --> FuenteDatos : persistencia
RepoCopro --> EntidadCopro : almacena / recupera
RepoEstado --> FuenteDatos : persistencia
RepoEstado --> EntidadEstado : almacena / recupera

note top of CtxBusqueda #FFF2CC
Strategy (REQ005):
Permite cambiar dinámicamente
la búsqueda de copropietarios
por casa, nombre, perfil o
estado de cuenta.
end note

note top of CtxReporte #FFF2CC
Strategy (REQ006):
Selecciona el algoritmo de
reporte (pagos, morosidad,
estado de cuenta) según el
tipo solicitado y el rol del
usuario autenticado.
end note

note bottom of Auditoria #FFF2CC
Decorator (REQ004, REQ007, REQ008):
AuditoriaCopropietarioDecorator y
ValidacionCopropietarioDecorator

```

envuelven al servicio base sin  
modificarlo, agregando auditoría  
y validación de datos/historial.  
end note

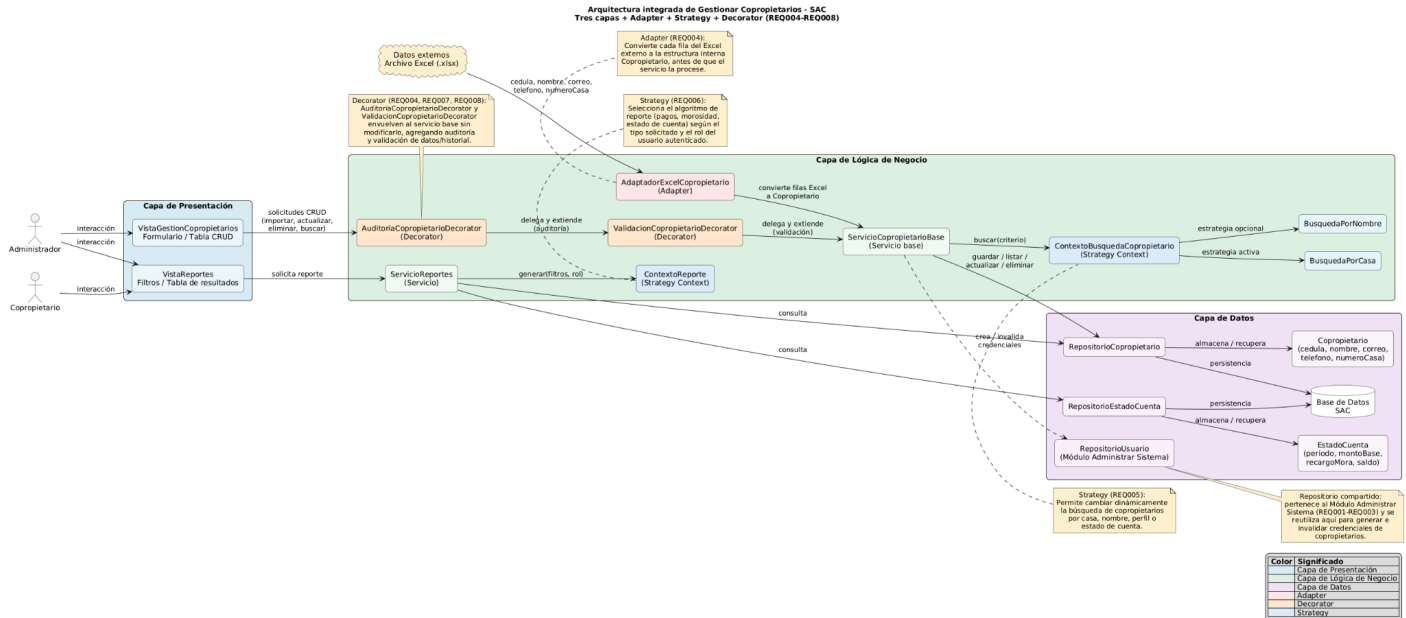
note bottom of Adapter #FFF2CC  
Adapter (REQ004):  
Convierte cada fila del Excel  
externo a la estructura interna  
Copropietario, antes de que el  
servicio la procese.  
end note

note right of RepoUsuario #FFF2CC  
Repositorio compartido:  
pertenece al Módulo Administrar  
Sistema (REQ001-REQ003) y se  
reutiliza aquí para generar e  
invalidar credenciales de  
copropietarios.  
end note

legend right  
|= Color |= Significado |  
<#D9EAF7>| Capa de Presentación |  
<#DFF2E1>| Capa de Lógica de Negocio |  
<#F3E0F7>| Capa de Datos |  
<#FFE6E6>| Adapter |  
<#FFE6CC>| Decorator |  
<#DCEBFA>| Strategy |  
endlegend

@enduml

## 2. Captura del Diagrama Generado



En conjunto los tres patrones viven dentro de la capa de Lógica de Negocio y refuerzan, no contradicen, la separación de capas : ni Adapter, ni Strategy, ni Decorator exponen detalles de implementación hacia la capa de Presentación ni hacia la Capa de Datos.